

Введение

В данной статье хотелось бы рассказать про конечные автоматы более-менее нормальным языком с большим числом примеров.

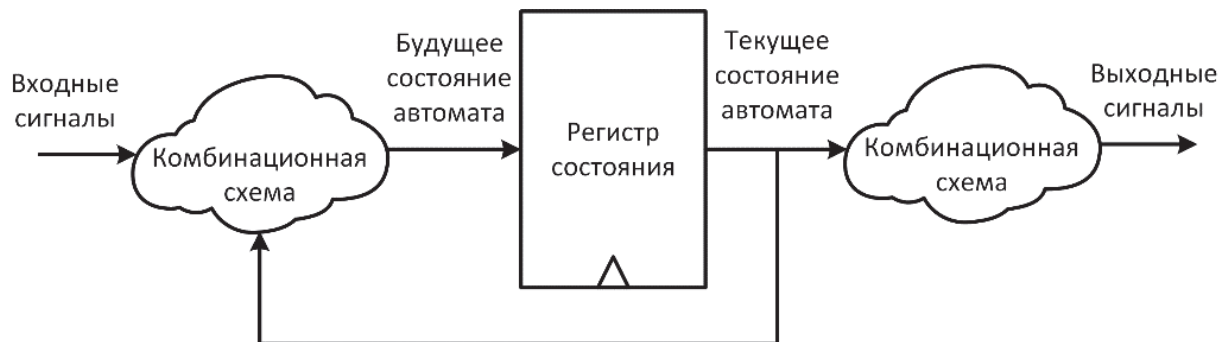
Конечные автоматы

По своей сути, конечный автомат – коробочка, которая умеет хранить хотя бы 1 бит информации (комбинационно преобразованный вход коробочки) и выдавать на свой выход или же преобразованное комбинационной схемой значение с выхода ячейки памяти (автомат Мура) или же преобразованное комбинационной схемой значение с выхода ячейки памяти и внешнего входа коробочки (автомат Мили).

Если же говорить про эту сущность простым языком, то это D-триггер/регистр с комбинационкой в обратной связи + комбинационные схемы входе и выходе схемы.

Автомат Мура.

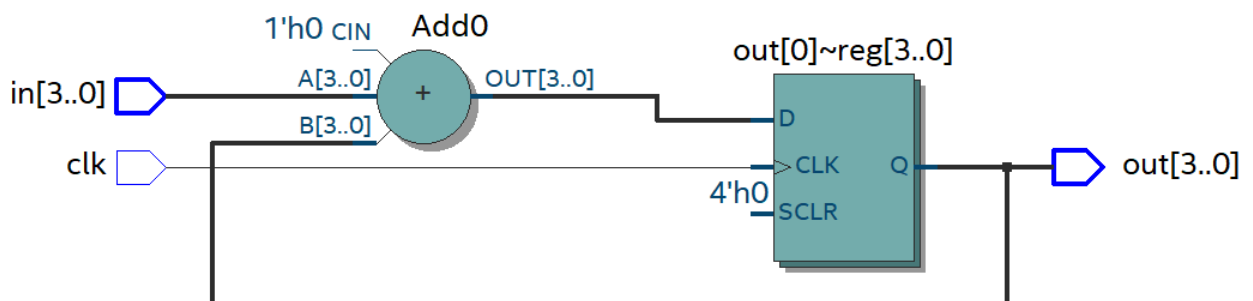
Концептуально (можете посмотреть на определение выше) автомат Мура в общем виде выглядит так:



Давайте теперь рассмотрим несколько схем и подумаем, являются ли они автоматами Мура или же нет.

Пример 1. Счетчик.

Для обоснования того, почему счетчик является конечным автоматом Мура достаточно сравнить его RTL схему с схемой выше.

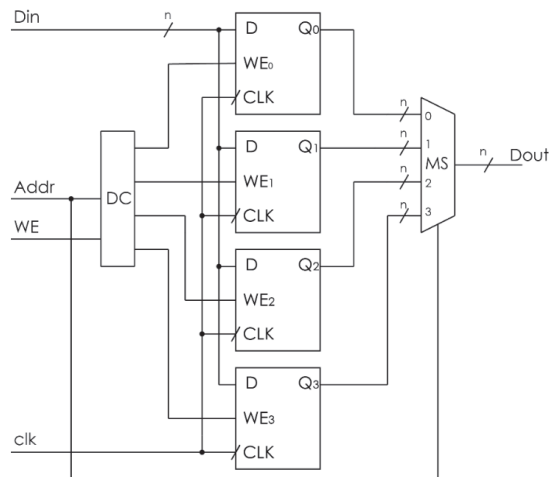


Как мы видим, роль входной комбинационной схемы играет сумматор, а роль выходной комбинационной схемы – просто провод, так что счетчик является конечным автоматом Мура.

Пример 2. Регистровый файл.

Как мы знаем, концептуально регистровый файл выглядит как на картинке ниже (в методичке по высшей цифровой схемотехнике на выходе были буферы с 3 состоянием + дешифратор, что являлось не самой лучшей реализацией, тут же их роль выполняет мультиплексор)

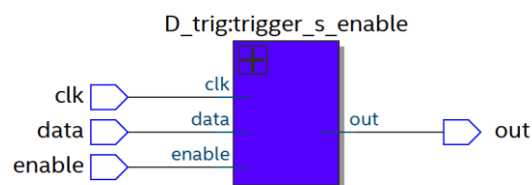
Концептуальная схема регистрового файла



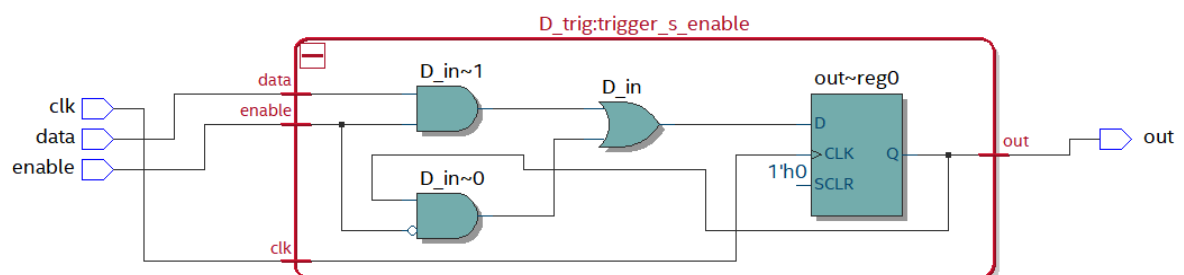
Почему же регистровый файл является автоматом Мура, если у него нет явного провода с выхода регистров на вход?

Однако, если вспомнить реализацию enable у D триггеров (смотри методичку по цифровой схемотехнике), то все встанет на свои места:

Схема D триггера с enable



Реализация enable



Тогда мы явно видим, что регистровый файл является в чистейшем виде автоматом Мура (с выхода регистра на вход комбинационная схема, на выходе тоже комбинационная схема (мультиплексор))

Зачем нужны конечные автоматы (классификация зачем)?

У читателя мог бы возникнуть резонный вопрос, что хорошо, мы обозвали некоторое множество схем конечными автоматами, для чего она нам нужна при разработке схемы на Verilog.

Однако, причинами именно реализация схемы именно как готового блока конечного автомата (желтая коробочка) в том же Quartus являются:

1) Задав алгоритмически все переходы в схеме, можно легко проверить правильность описания на Verilog по таблице перехода/графу перехода, который сгенерируется по описанию на Verilog (сравнить их с теми, которые ручками построили).

2) Предсказуемость синтеза (обычно желтые коробочки оптимально синтезируются)

3) Удобно описывать сложные системы из-за графа переходов

То есть, можно сказать, что конечные автоматы – метод проектирования устройств на Verilog

Пример 3. Прикольный счетчик.

Давайте в терминах конечных автоматов построим счетчик, который при $in = 1$ и по clk делает $+1$, иначе не меняется, а когда доходит до 2, то обнуляется

Пусть у нашего счетчика есть 3 состояния:

$S0 = 2'd0$

$S1 = 2'd1$

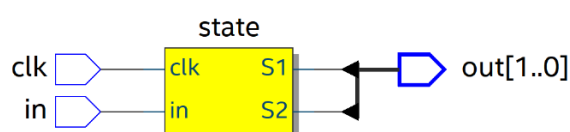
$S2 = 2'd2$

Если $in = 1$, то мы переходим в следующее состояние, что можно представить в виде таблицы:

Можно составить таблицу переходов для данной схемы:

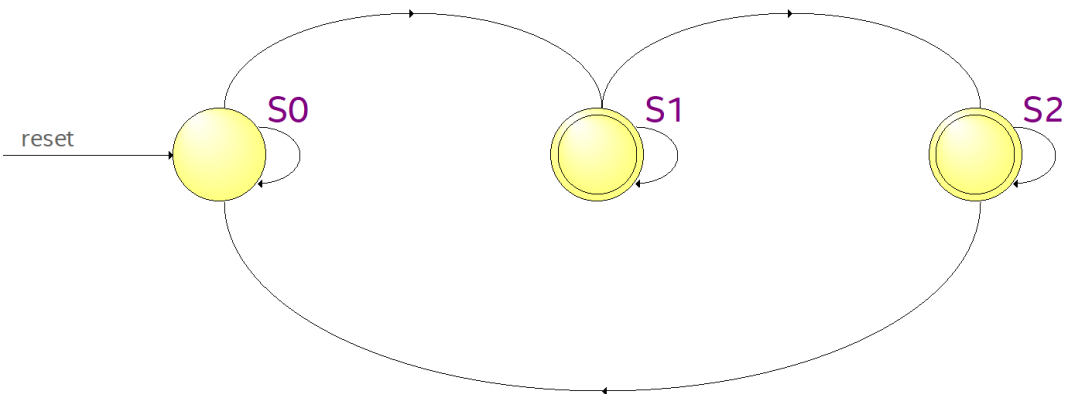
Состояние автомата	Следующее состояние	Условие перехода
S0	S1	$in = 1$
S0	S0	$in = 0$
S1	S2	$in = 1$
S1	S1	$in = 0$
S2	S0	$in = 1$
S2	S2	$in = 0$

RTL схема конечного автомата



Если 2 раза нажмем на желтую коробочку, то появится диаграмма переходов

Диаграмма переходов конечного автомата (сгенерирована Quartus)



Как мы видим, диаграмма переходов является просто графическим отображением таблицы переходов (например, если in = 1, то S0 переходит в S1, если in = 0, то S0 переходит в S0).

Описание на Verilog схемы как конечного автомата

```
1 module Counter (
2     input in,
3     input clk,
4     output [1:0] out
5 );
6
7 reg [1:0] state;
8 reg [1:0] next_state;
9 assign out = state;
10 initial state = S0;
11
12 localparam S0 = 2'd0,
13             S1 = 2'd1,
14             S2 = 2'd2;
15
16 always @(*) begin
17     case(state)
18     S0: next_state = (in) ? S1 : S0;
19     S1: next_state = (in) ? S2 : S1;
20     S2: next_state = (in) ? S0 : S2;
21     default: next_state = S0;
22     endcase
23 end
24
25 always @(posedge clk) begin
26     state <= next_state;
27 end
28
29 endmodule
```

Таблица переходов, которую сгенерировал Quartus.

Переход происходит в данной табличке при Condition = 1, так что читатель может сравнить таблицу, которую нарисовали вручную с тем, что сгенерировалось и проверить правильность кода (в этом, собственно, один из основных плюсов реализации каких-либо схем именно через встроенный блок конечного автомата)

	Source State	Destination State	Condition
1	S0	S1	(in)
2	S0	S0	(!in)
3	S1	S2	(in)
4	S1	S1	(!in)
5	S2	S2	(!in)
6	S2	S0	(in)

Пример 4. Задача из Школы Синтеза про конечные автоматы

Ознакомиться с включением обнаружения 4-х битной последовательности.

Задание: Реализовать модуль для детектирования 6-ти битной последовательности 110011 с помощью конечного автомата.

Хотя в данном задании проще реализовать конечный автомат “вручную” без коробочек (например, сдвиговый регистр с “И” в роли ключей на выходах), но раз уж просят рассмотреть данную задачу как абстрактный конечный автомат в общем виде (чтобы появилась желтая коробочка и граф переходов), то давайте его делать.

Шаг 1. Принцип детектирования последовательности.

Заведем некие состояния схемы, переключаться между которыми она будет благодаря входному биту а, и сделать так, чтобы последнее состояние достигалось при последовательности входной 110011. Смысл состояний – сколько битов у последовательности подряд соответствует 110011.

Perv_yes (1) = 1

Vtor_yes (2) = 11

Tret_yes (3) = 011

Chetv_yes (4) = 0011

Pyat_yes (5) = 10011

Six_yes (full) = 110011

Шаг 2. Формализация принципа.

Входной бит а	Текущее состояние (сколько битов совпадает с последовательностью)	Следующее состояние (сколько будет совпадать с входным битом а)
0	IDLE (0)	IDLE (0)
1	IDLE (0)	Perv_yes (1)
0	Perv_yes (1)	IDLE (0)
1	Perv_yes (1)	Vtor_yes (2)
0	Vtor_yes (2)	Tret_yes (3)
1	Vtor_yes (2)	Vtor_yes (2)
0	Tret_yes (3)	Chetv_yes (4)
1	Tret_yes (3)	IDLE (0)
0	Chetv_yes (4)	IDLE (0)
1	Chetv_yes (4)	Pyat_yes (5)
0	Pyat_yes (5)	Perv_yes (1)
1	Pyat_yes (5)	Six_yes (полностью совпадает число, на выход подаем 1)

RTL схема конечного автомата

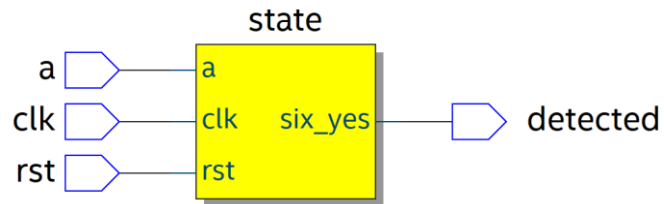
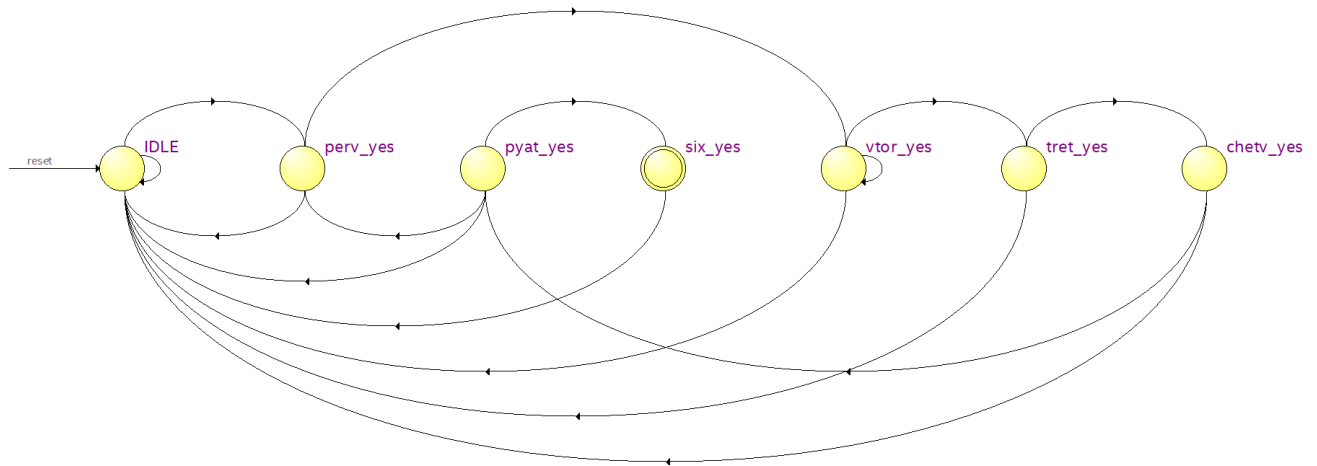


Диаграмма переходов конечного автомата (сгенерирована Quartus)



Описание на Verilog.

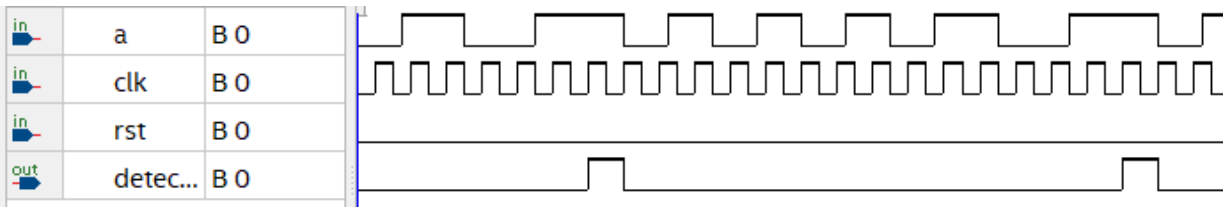
```
1 module Poisk (
2     input clk,
3     input rst,
4     input a,
5     output detected
6 );
7
8     localparam IDLE = 3'd0,
9                 perv_yes = 3'd1,
10                 vtor_yes = 3'd2,
11                 tret_yes = 3'd3,
12                 chetv_yes = 3'd4,
13                 pyat_yes = 3'd5,
14                 six_yes = 3'd6;
15
16     reg [2:0] state;
17     reg [2:0] new_state;
18
19     always @(*) begin
20         case (state)
21             IDLE: new_state = (a) ? perv_yes : IDLE;
22             perv_yes: new_state = (a) ? vtor_yes : IDLE;
23             vtor_yes: new_state = (~a) ? tret_yes : vtor_yes;
24             tret_yes: new_state = (~a) ? chetv_yes : IDLE;
25             chetv_yes: new_state = (a) ? pyat_yes : IDLE;
26             pyat_yes: new_state = (a) ? six_yes : perv_yes;
27             default: new_state = IDLE;
28         endcase
29     end
30
31     always @(posedge clk)
32         if (rst)
33             state <= IDLE;
34         else
35             state <= new_state;
36
37     assign detected = (state == six_yes);
38 endmodule
```

Таблица переходов конечного автомата (сгенерирована Quartus)

	Source State	Destination State	Condition
1	chetv_yes	pyat_yes	(a).(!rst)
2	chetv_yes	IDLE	(!a) + (a).(rst)
3	IDLE	perv_yes	(a).(!rst)
4	IDLE	IDLE	(!a) + (a).(rst)
5	perv_yes	vtor_yes	(a).(!rst)
6	perv_yes	IDLE	(!a) + (a).(rst)
7	pyat_yes	six_yes	(a).(!rst)
8	pyat_yes	perv_yes	(!a).(!rst)
9	pyat_yes	IDLE	(rst)
10	six_yes	IDLE	
11	tret_yes	IDLE	(!a).(rst) + (a)
12	tret_yes	chetv_yes	(!a).(!rst)
13	vtor_yes	vtor_yes	(a).(!rst)
14	vtor_yes	tret_yes	(!a).(!rst)
15	vtor_yes	IDLE	(rst)

Как читатель видит, таблица совпадает с построенной вручную

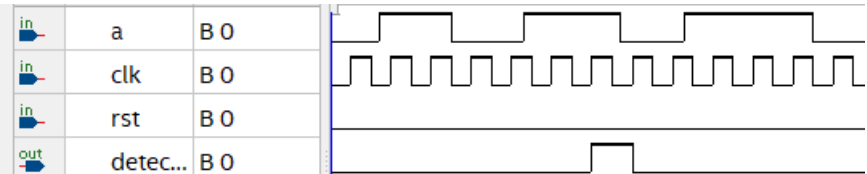
Тестирование схемы



Как читатель видит, при последовательности 110011 схема выдает 1

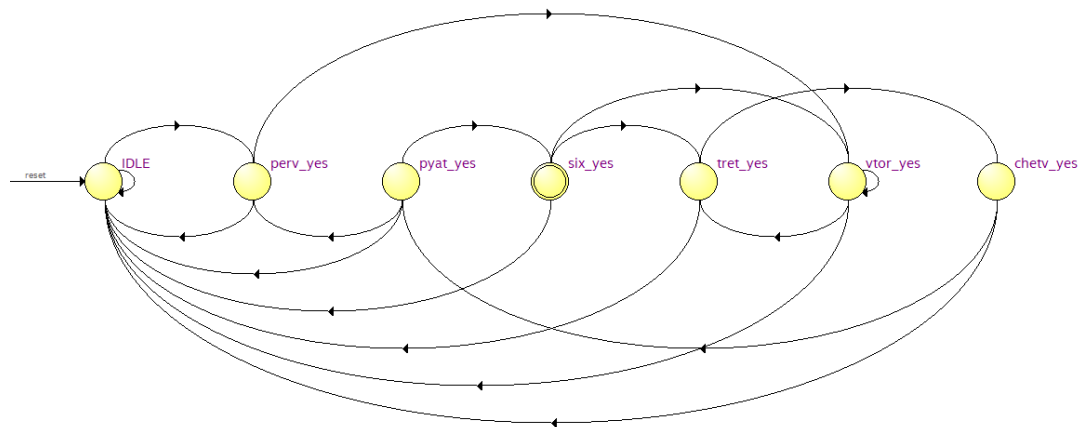
Недоработки схемы (можно было бы 6 состояние чутко подробнее описать, что оставляю на самостоятельное развлекало читателям).

Из-за этого, например, в последовательности 1100110011 “110011” обнаружит автомат только 1 раз.

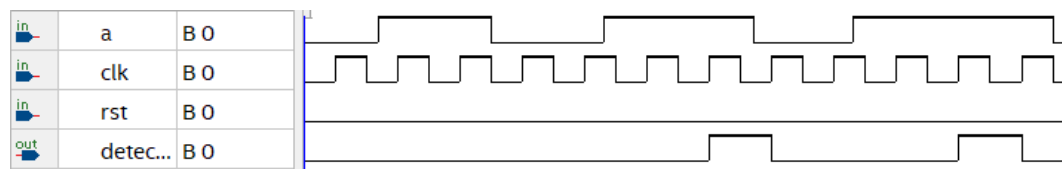


Чуток поменяв код можно получить:

Новый граф переходов



Результат симуляции



Как читатель видит, теперь с данным тестом автомат справляется.

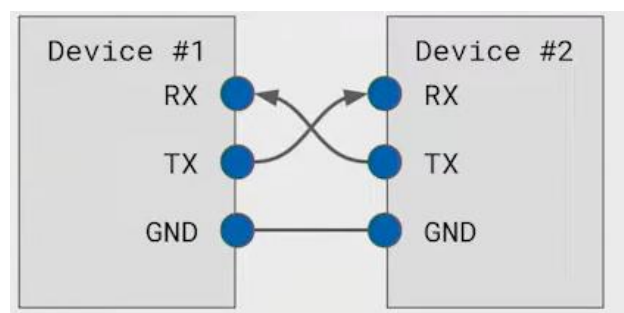
UART интерфейс.

Данная штука встроена, например, в ту же Arduino и позволяет ему общаться с компьютером (COM-порт – это UART интерфейс).

Как и все интерфейсы, является данная штука просто аппаратной реализацией правил передачи данных (а правила = таблица переходов конечного автомата, который и синтезируется для данного интерфейса). Давайте теперь поговорим про правила сначала общим языком, а потом уже формализуем их (таблицу переходов сделаем).

Теория про UART.

UART (Universal Asynchronous Receiver-Transmitter) – последовательный интерфейс для асинхронной передачи данных. Данный протокол обмена данными представляет собой 2 одинаковые коробочки с отдельными входами для передачи и приема данных (и с общей землей)

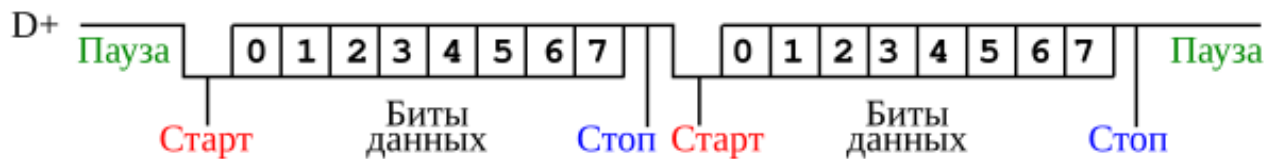


RX – отвечает за прием данных

TX – отвечает за передачу данных

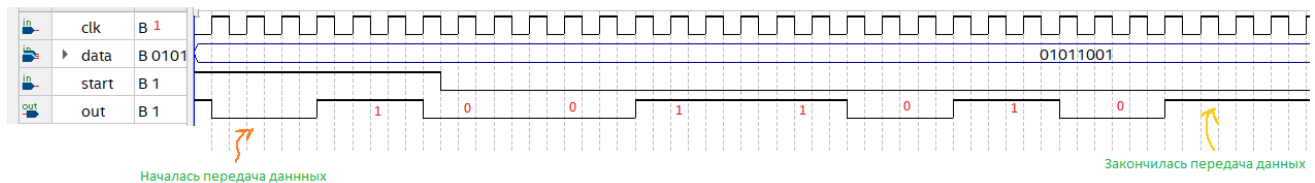
Как читатель видит, данные передаются как от 1 устройства ко 2, так и от 2 к 1 довольно симметрично. Асинхронным же называется данный протокол из-за отсутствия явного clk (на самом деле, как мы увидим ниже, передача данных привязана к частоте clk, но нет привязке к event (например, перехода clk из 0 в 1) и частота передачи/приема данных на каждом устройстве должна быть выставлена отдельно (аппаратно надо одинаковым образом привязать одинаково частоту передачи данных f к clk общему).

Передача данных UART



В UART для начала передачи данных (пускай они передаются по 1 проводу, начало передачи по сигналу start) мы сначала на выходной провод выставляем 0, а потом в заранее определенные промежутки времени (время измеряется относительно clk) закидываем побитово данные, а после передачи данных (байт, 2 байта, 4 байта, заранее определяется при разработке возможный размер) выставляем на выход 1.

Пример работы передатчика UART

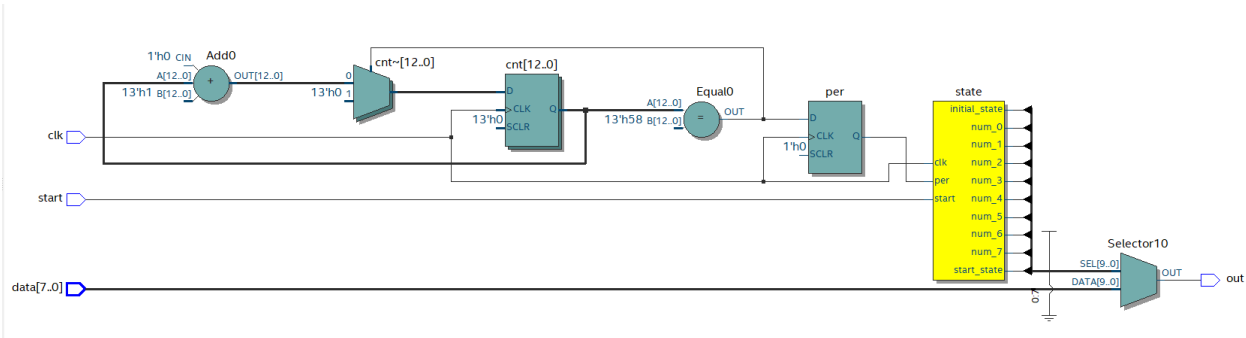


Рассмотрим процесс передачи информации как конечный автомат – когда счетчик доходит до некоего определенного значения, то он характерный бит делает = 1, и у нас автомат будет переходить на следующее состояние (как раз привязка к clk и к частоте передачи). Состоянием же автомата будут биты данных (какой бит data передается на выход, после него по событию (per) передавать следующий бит, иначе не менять). Также, введем режим ожидания (initial_state, когда out = 1) и начальную подачу 0 на выход (start_state) как отдельные состояния. Тогда, по вышеописанному алгоритму работы передачи UART можно получить такую таблицу переходов:

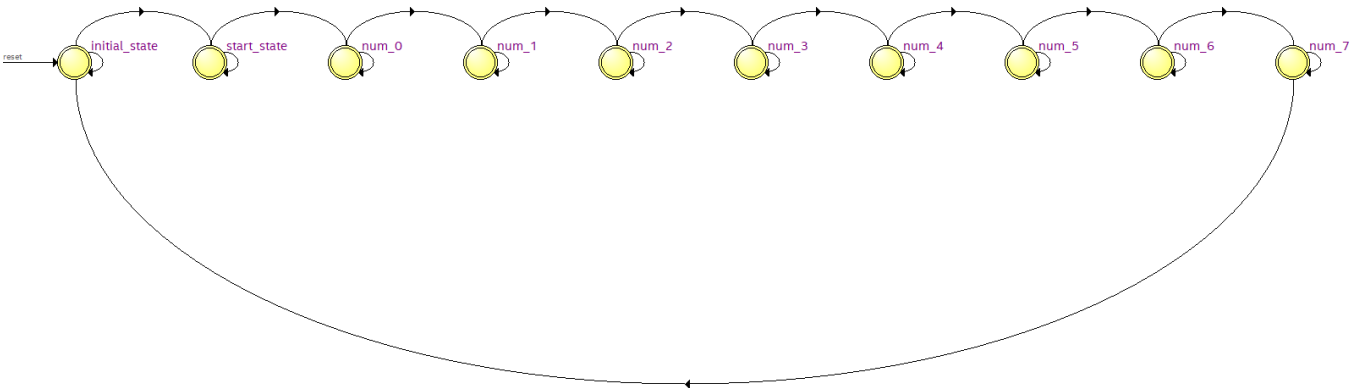
Таблица переходов для передачи UART.

	Source State	Destination State	Condition
1	initial_state	initial_state	(!start)
2	initial_state	start_state	(start)
3	num_0	num_0	(!per)
4	num_0	num_1	(per)
5	num_1	num_1	(!per)
6	num_1	num_2	(per)
7	num_2	num_2	(!per)
8	num_2	num_3	(per)
9	num_3	num_3	(!per)
10	num_3	num_4	(per)
11	num_4	num_4	(!per)
12	num_4	num_5	(per)
13	num_5	num_5	(!per)
14	num_5	num_6	(per)
15	num_6	num_6	(!per)
16	num_6	num_7	(per)
17	num_7	initial_state	(per)
18	num_7	num_7	(!per)
19	start_state	num_0	(per)
20	start_state	start_state	(!per)

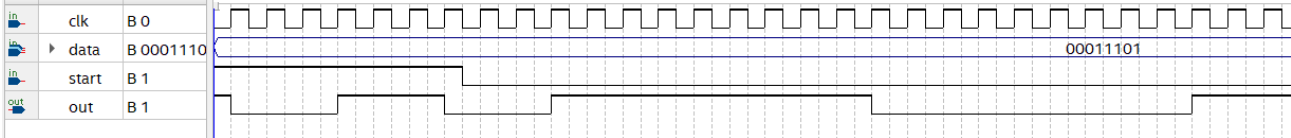
RTL схема



Граф переходов (синтезирован Quartus)



Тестирование передатчика UART.



Напомню, что передается число “задом наперед”, но, так-как приемник принимает тоже “задом наперед”, то в итоге число передается нормально.

Итак, мы передаем 00011101 (как мы поняли, реально 10111000), новый бит data идет каждые 4 clk. Вначале на выход подается ноль, как сигнал для приемника о записи данных, затем идут сами данные (читатель может по рисунку понять, что это 10111000), а после этого на out снова ставится 1, как признак конца сообщения (приемник и передатчик настроены на одну и ту же частоту и одно и тоже число получаемых/передаваемых битов, поэтому последнюю единицу приемник должен определить как конец считывания данных с передатчика)

Описание на Verilog (код есть на GitHub).

```
1 module my_UART (  
2     input [7:0] data,  
3     input start,  
4     input clk,  
5     output reg out  
6 );  
7  
8 localparam NUM_CLK = 13'd2;  
9 reg [12:0] cnt = 13'd0;  
10 reg per = 1'd0;  
11  
12 always @(posedge clk) begin  
13     if (cnt == NUM_CLK) begin  
14         cnt <= 13'd0;  
15         per <= 1'd1;  
16     end  
17     else begin  
18         cnt <= cnt + 13'd1;  
19         per <= 1'd0;  
20     end  
21 end  
22  
23 reg [3:0] state = initial_state;  
24 reg [3:0] next_state;  
25 localparam  
26     initial_state = 4'd0,  
27     num_0 = 4'd1,  
28     num_1 = 4'd2,  
29     num_2 = 4'd3,  
30     num_3 = 4'd4,  
31     num_4 = 4'd5,  
32     num_5 = 4'd6,  
33     num_6 = 4'd7,  
34     num_7 = 4'd8,  
35     start_state = 4'd9;  
36  
37 always @(*) begin  
38     case (state)  
39         initial_state: next_state = (start) ? start_state : initial_state;  
40         start_state : next_state = (per) ? num_0 : start_state;  
41         num_0 : next_state = (per) ? num_1 : num_0;  
42         num_1 : next_state = (per) ? num_2 : num_1;  
43         num_2 : next_state = (per) ? num_3 : num_2;  
44         num_3 : next_state = (per) ? num_4 : num_3;  
45         num_4 : next_state = (per) ? num_5 : num_4;  
46         num_5 : next_state = (per) ? num_6 : num_5;  
47         num_6 : next_state = (per) ? num_7 : num_6;  
48         num_7 : next_state = (per) ? initial_state : num_7;  
49     endcase  
50  
51     case (state)  
52         initial_state : out = 1'd1;  
53         start_state : out = 1'd0;  
54         num_0 : out = data[0];  
55         num_1 : out = data[1];  
56         num_2 : out = data[2];  
57         num_3 : out = data[3];  
58         num_4 : out = data[4];  
59         num_5 : out = data[5];  
60         num_6 : out = data[6];  
61         num_7 : out = data[7];  
62     endcase  
63 end  
64  
65 always @(posedge clk) begin  
66     state <= next_state;  
67 end  
68  
69 endmodule
```

Кстати, забавно то, что изначально я делал время передачи данных для 3 clk (задержка на счетчике), но не учел того, что между счетчиком и автоматом синтезировался регистр, поэтому добавился 1 clk к длине передачи данных (то есть, стало 4 clk на бит data).

Небольшая задачка

Однако, вариант выше был скорее демонстрационным, так-как существуют некие стандарты UART по скорости передачи данных (9600 бит/с, 115200 бит/с, 57600 бит/с и тд).

Также, мы знаем, что у ПЛИС Cyclone-IV, Cyclone-V есть встроенный генератор clk на 50 МГц (который мы и будем использовать). До сколько нужно считать счетчику, чтобы реализовать передачу данных 9600 бит/с ?

Решение:

$$\frac{f_{clk}}{N} = f_{UART} \text{ — это из — за счетчика, который до } N \text{ считает}$$

$$N = \frac{f_{clk}}{f_{UART}} = \frac{50 * 10^6}{9600} = 5208 \text{ — до этого числа — 2 должен считать счетчик}$$

(так-как счетчик считает с 0 и у нас есть задержка на 1 clk в буферном регистре).

То есть, мы просто поставим NUM_CLK = 13'd5206

Почему же я поставил другое число? Это из-за того, что симулятор, встроенный в Quartus может моделировать до 100 мкс, а этого времени недостаточно для передачи данных с скоростью 9600 бит/с, поэтому я и поставил время передачи одного бита как 4 clk.

UART приемник.

В общем-то, конечный автомат, что мы реализовали в передатчике UART оставляем прежним (так-как его состояния – биты передаваемой data, что как раз и принимает приемник).

Поменять лишь стоит сам case, который эти состояния расшифровывает (что теперь мы записываем данные в шину 8-битную приемника data).

Как надо поменять case

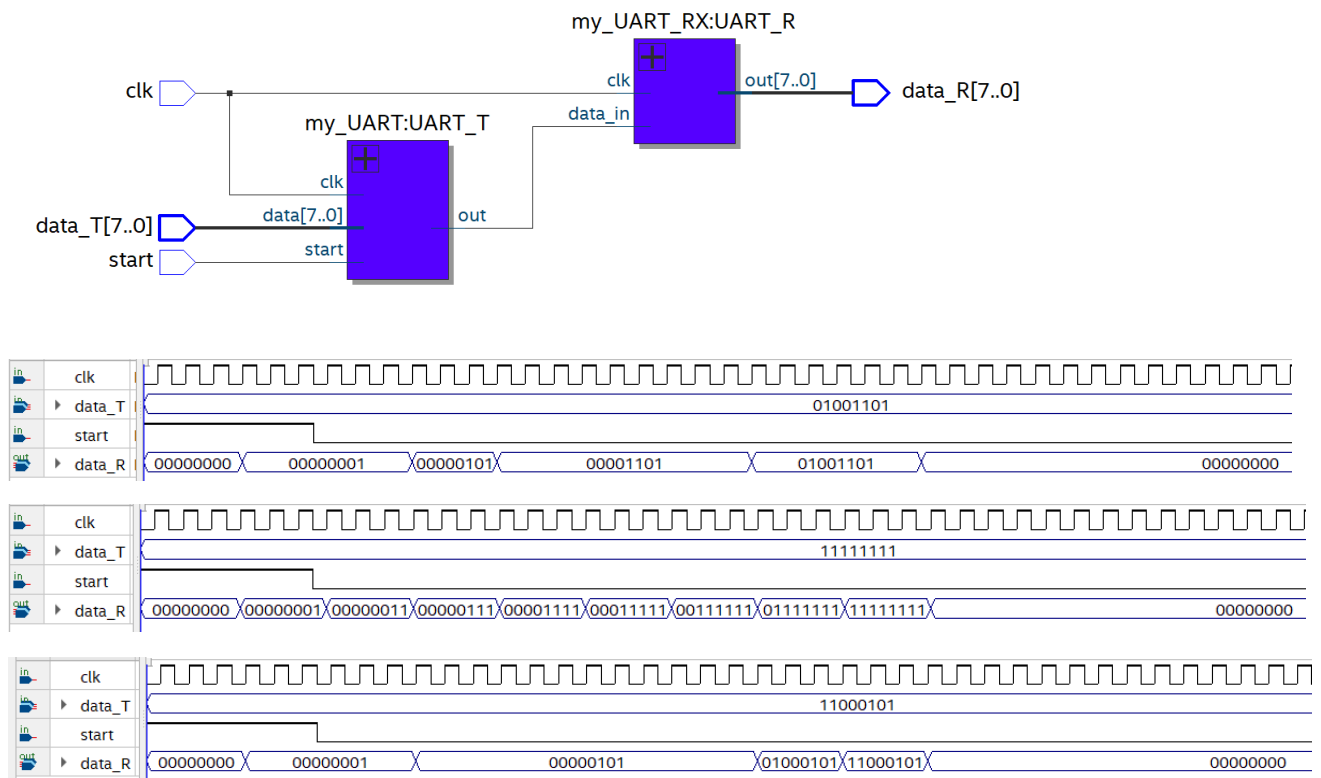
```
case (state)
  num_0 : out[0] = data_in;
  num_1 : out[1] = data_in;
  num_2 : out[2] = data_in;
  num_3 : out[3] = data_in;
  num_4 : out[4] = data_in;
  num_5 : out[5] = data_in;
  num_6 : out[6] = data_in;
  num_7 : out[7] = data_in;
  default : out = 8'd0;
endcase
```

И, да, у приемника теперь вход однобитный, а выход – 8-битный (вот тут вот уже можно играть с тем, что выход приемника записывает в какую-то память, добавлять некие характерные биты и тд, я же просто реализовал шину, в которую побитово приходят данные (очень похожую на стек))

Входы и выходы модули приемника UART

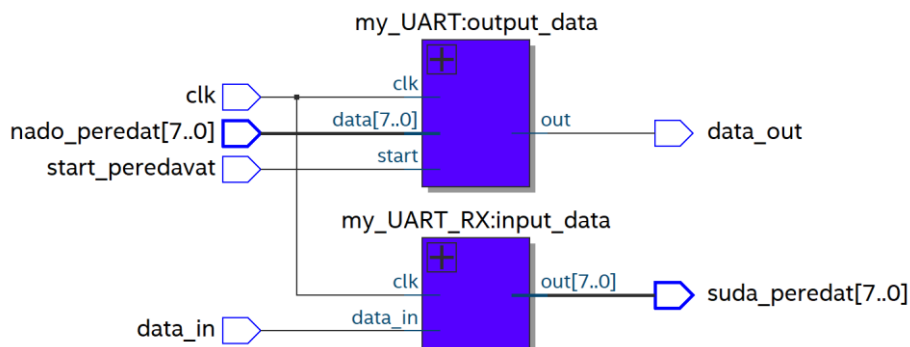
```
module my_UART_RX (
  input data_in,
  input clk,
  output reg [7:0] out
);
```

Проверка UART передачи данных (сделал передачу и прием данных сам себе)



Разумеется, реальный UART чуток посложнее (там есть та же проверка правильной данных, возможность переключения между режимами (8 бит, 16 бит, 32 бит пакет) и тому подобное), но в качестве учебного и демонстрационного варианта данный пример передает суть реального UART без буквоядства.

Ура, мы собрали волшебную коробочку UART на конечных автоматах!



Nado_peredat – 8-битный стек, с которого берем и передаем данные

Suda_peredat – 8-битный стек, в который мы закидываем полученные данные

Заключение

Что-ж, надеюсь, после прочтения данной статьи читателям стало более-менее понятно как сама концепция конечного автомата, так и способ его построения/применения в реальных устройствах (потому как я хоть и разобрал только UART, но для других интерфейсов способ построения похожий).